

Final Year Design Project System Requirements Specification

IoT-Based Smart Home Automation System Using ESP32

Final Year Design Project SRS

by

M. Mohsin Saleem	BCS22006
Abdul Rehman Tahir	BCS22016
Rehan Ali	BCS22021
Muhammad Owais	BCS22022
Wasif Ali	BCS22024

Project Advisor

Dr. Salman Naseer

Department of Information Technology

University of the Punjab, Gujranwala Campus, Gujranwala, Pakistan

(2025)

IoT-Based Smart Home Automation System Using ESP32

Executive Summary

*This System Requirements Specification (SRS) defines the functionality, behaviour, and constraints of the **IoT-Based Smart Home Automation System using ESP32**. The system enables users to control Wi-Fi-enabled home appliances through a unified mobile application built with Flutter. Core features include remote ON/OFF control, secure authentication through Firebase, and real-time synchronization of device states.*

*Beyond manual control, the system introduces an **Automatic Mode** that activates intelligent behaviour based on sensor inputs. The ESP32 processes data from temperature, motion, light-level, and water-level sensors to automate appliances such as fans, lights, curtains, and water pumps. These automations enhance convenience, improve energy efficiency, and reduce manual effort. A peak-time buzzer alert is also integrated to notify users during predefined time intervals.*

The system architecture ensures seamless communication between the mobile app, Firebase backend, and ESP32 hardware, enabling reliable device control and real-time updates. This document outlines the system's functional and non-functional requirements, interaction flows, and operational limitations, forming a clear foundation for the design and implementation phases of the project.

Table of Contents

<i>Executive Summary</i>	<i>ii</i>
1. Requirements Analysis	6
1.1 User Classes and Characteristics	6
1.2 Requirement Identifying Technique	6
1.3 Functional Requirements	7
1.3.1 User Registration (Functional Requirement: FR-1)	7
1.3.2 User Login (Functional Requirement: FR-2)	8
1.3.3 Device Onboarding (Functional Requirement: FR-3)	8
1.3.4 Remote Appliance ON/OFF Control (Functional Requirement: FR-4)	9
1.3.5 Automatic Mode Activation (Functional Requirement: FR-5)	10
1.3.6 Temperature-Based Fan Control (Functional Requirement: FR-6)	10
1.3.7 Motion-Based Light Automation (Functional Requirement: FR-7)	11
1.3.8 Light-Level Curtain Automation (Functional Requirement: FR-8)	11
1.3.9 Water-Level Pump Automation (Functional Requirement: FR-9)	12
1.3.10 Peak-Time Buzzer Alert (Functional Requirement: FR-10)	12
1.3.11 Communication Handling (Functional Requirement: FR-11)	13
1.3.12 Subscription Management (Functional Requirement: FR-12)	14
1.4 Non-Functional Requirements	14
1.4.1 Reliability	14
1.4.2 Usability	15
1.4.3 Performance	15
1.4.4 Security	15
1.5 External Interface Requirements	16
1.6 User Interfaces Requirements	16
1.7 Software Interfaces	16
1.8 Hardware Interfaces	17
1.9 Communications Interfaces	17
2. Use Case Analysis	18
2.1 Use Case #1: User Registration (FR-1)	18
2.2 Use Case #2: Login to the System (FR-2)	19
2.3 Use Case #3: Add/Pair a New Device (ESP32) (FR-3)	20
2.4 Use Case #4: Control Device ON/OFF (FR-4)	21
2.5 Use Case #5: Activate Automatic Mode (FR-5)	22
2.6 Use Case #6: Temperature-Based Fan Automation (FR-6)	23

2.7	Use Case #7: Motion-Based Light Automation (FR-7).....	24
2.8	Use Case #8: Light-Level Curtain Automation (FR-8)	25
2.9	Use Case #9: Water-Level Pump Automation (FR-9)	26
2.10	Use Case #10: Peak-Time Buzzer Alert (FR-10)	27
2.11	Use Case #11: View Device Status (FR-11).....	28
2.12	Use Case #12: Update Device Settings (FR-5, FR-6, FR-7, FR-8, FR-9) 29	
2.13	Use Case #13: Manage Subscription (FR-12).....	30
2.14	Use Case Diagram	30
3.	<i>Storyboards</i>	32
3.1	Storyboard 1: Registration, Login, and Dashboard Flow	32
3.2	Storyboard 2: Device Pairing and Control	32
3.3	Storyboard 3: Fan Automation (Temperature Sensor)	7
4.	<i>Summary</i>	6
5.	<i>References</i>	7

List of Tables

Table 1: User Class and Characteristics.....	6
Table 2: Functional Requirement - User Registration	7
Table 3: Functional Requirement - User Login	8
Table 4: Functional Requirement - Device Onboarding (Pairing).....	8
Table 5: Functional Requirement - Remote Appliance ON/OFF Control	9
Table 6: Functional Requirement – Automatic Mode Activation	10
Table 7: Functional Requirement – Temperature Based Fan Control	10
Table 8: Functional Requirement – Motion-Based Light Automation.....	11
Table 9: Functional Requirement – Light-Level Curtain Automation	11
Table 10: Functional Requirement – Water-Level Pump Automation.....	12
Table 11: Functional Requirement – Peak-Timer Buzzer Alert	12
Table 12: Communication Handling.....	13
Table 13: Subscription management.....	14
Table 14: Use Case Description for "User Registration"	18
Table 15: Use Case Description for " Login to the System "	19
Table 16: Use Case Description for "Adding/Pairing a new appliance"	20
Table 17: Use Case Description for " Controlling Device "	21
Table 18: Use Case Description for " Automatic Mode Activation "	22
Table 19: Use Case Description for " Temperature-Based Fan Automation "	23
Table 20: Use Case Description for " Motion-Based Light Automation "	24
Table 21: Use Case Description for " Light-Level Curtain Automation "	25
Table 22: Use Case Description for " Water-Level Pump Automation "	26
Table 23: Use Case Description for " Peak-Time Alert "	27
Table 24: Use Case Description for " View Device Status"	28
Table 25: Use Case Description for "Update Device Settings"	29
Table 26: Use Case Description for " Manage Subscription ".....	30

List of Figures

Figure 1: User Registration (FR-1)	18
Figure 2: Login to the System (FR-2).....	19
Figure 3: Add/Pair an Appliance (FR-3)	20
Figure 4: Control Device ON/OFF (FR-4)	21
Figure 5: Automatic Mode Activation (FR-5).....	22
Figure 6: Temperature-Based Fan Automation (FR-6)	23
Figure 7: Motion-Based Light Automation (FR-7)	24
Figure 8: Light-Level Curtain Automation (FR-8).....	25
Figure 9: Water-Level Pump Automation (FR-9)	26
Figure 10: Peak-Time Alert (FR-10)	27
Figure 11: View Device Status (FR-11)	28
Figure 12: Update Device Settings	29
Figure 13: Manage Subscription (FR-12)	30
Figure 14: Use Case Diagram	31

1. Requirements Analysis

This section defines what the smart home automation system is expected to do and under which conditions it must operate. It identifies user needs, system behaviour, and the constraints that guide the design and implementation phases. The goal is to translate the project concept into clear, actionable requirements that ensure the system delivers convenience, reliability, and energy-efficient automation.

The system integrates **Wi-Fi-enabled appliances**, **ESP32-based sensor automation**, and a **Flutter mobile application** connected through **Firebase**. Users can remotely control devices, monitor real-time status, and enable Automatic Mode, where the ESP32 makes intelligent decisions based on sensor inputs such as temperature, motion, light level, and water level.

1.1 User Classes and Characteristics

This section identifies the primary user groups who will interact with the system. Understanding these users helps shape the interface design, automation features, and overall usability. Since the system relies on mobile control and cloud-based communication, it supports a wide range of users with varying technical backgrounds.

Table 1: User Class and Characteristics

User Class	User Characteristics
Homeowner	Sets up devices, manages automation modes, and monitors home appliances through the mobile app.
Family Member	Daily user who controls appliances through the mobile app; no technical setup required.
Renter	Uses plug-and-play Wi-Fi devices and prefers simple, portable automation without complex installation.
Tech Enthusiast	Interested in experimenting with automation features, sensor-based behaviour, and custom ESP32 configurations.
Small Business Owner	Uses the system for basic lighting, fan, or pump control with optional automation for energy savings.

1.2 Requirement Identifying Technique

The requirements for the system are gathered using structured techniques to ensure clarity, completeness, and alignment with real-world usage. Since the system supports mobile control and sensor-based automation, a systematic approach is essential.

1.2.1 Primary Technique: Use Cases

Use cases describe how users interact with the system to achieve specific goals, such as controlling a device, enabling Automatic Mode, or viewing real-time

status. They provide a clear, scenario-based understanding of system behaviour and are widely used for communicating requirements among stakeholders. Use cases remain a widely adopted approach for requirement structuring due to clarity in stakeholder communication (Sommerville, 2016).

1.2.2 Supporting Technique: User Stories

User stories capture requirements from the user’s perspective in simple, natural language (e.g., “*As a user, I want to automatically turn on the fan when the room gets hot.*”). These stories help ensure the system remains practical, intuitive, and aligned with everyday usage patterns. They also support interface design and automation logic development. User-story method encourages requirement realism focused on everyday usage (Sommerville, 2016).

1.3 Functional Requirements

This section defines the functional behaviour of the IoT-Based Smart Home Automation System. After registering and logging in through Firebase Authentication, users can control all Wi-Fi-enabled appliances connected to the ESP32, monitor real-time device states, and enable Automatic Mode for sensor-based automation. The system follows modern cloud-connected smart-home models, enabling remote control, real-time synchronization, and intelligent automation. Features align with cloud-connected smart-home models enabling remote appliance control and energy automation (Ahmad, 2023; Sharma, 2022; Nguyen, Patel, & Kumar, 2025).

1.3.1 User Registration (Functional Requirement: FR-1)

This feature allows a new user to create an account using Firebase Authentication. As shown in **Table 2**, the system verifies the entered information and creates a unique user profile if all conditions are met.

Table 2: Functional Requirement - User Registration

Identifier	FR-1
Title	User Registration
Requirement	The system shall allow a new user to create an account by providing a valid email address and password through Firebase Authentication.
Source	Project Objective: Provide secure user registration and login through Firebase Authentication.
Rationale	Ensures that only authenticated users can access and control smart devices.

Business Rule (if required)	Password must be at least 8 characters long and contain at least one letter and one number.
Dependencies	--
Priority	High

1.3.2 User Login (Functional Requirement: FR-2)

This feature enables registered users to access their account using correct login credentials. As shown in **Table 3**, the system verifies details before granting access.

Table 3: Functional Requirement - User Login

Identifier	FR-2
Title	User Login
Requirement	The system shall allow a registered user to log in using Firebase Authentication by entering a valid email and password.
Source	Derived from FR-1
Rationale	Ensures secure access to device control and automation features.
Business Rule	After 3 failed attempts, the account shall be locked for 5 minutes.
Dependencies	FR-1 (User Registration)
Priority	High

1.3.3 Device Onboarding (Functional Requirement: FR-3)

This requirement enables users to add ESP32-controlled appliances to their account. As shown in **Table 4**, devices must be connected to the same Wi-Fi network during onboarding.

Table 4: Functional Requirement - Device Onboarding (Pairing)

Identifier	FR-3
Title	Device Onboarding (Wi-Fi Devices)

Requirement	The system shall allow a logged-in user to add ESP32-controlled appliances through a guided setup process.
Source	Project Objective: Ensure seamless communication between the mobile app, Firebase, and ESP32 hardware.
Rationale	Devices must be registered before they can be controlled or automated.
Business Rule	Devices must be connected to the same Wi-Fi network during onboarding.
Dependencies	FR-2 (User Login)
Priority	High

1.3.4 Remote Appliance ON/OFF Control (Functional Requirement: FR-4)

This requirement allows users to manually control appliances through the mobile app. Commands are sent to the ESP32 via Firebase. As shown in **Table 5**, the command is sent.

Table 5: Functional Requirement - Remote Appliance ON/OFF Control

Identifier	FR-4
Title	Remote Appliance Control (ON/OFF)
Requirement	The system shall allow the user to turn any connected ESP32-controlled appliance ON or OFF using the mobile application.
Source	Project Objective: Implement remote ON/OFF control for all connected devices.
Rationale	Core functionality for smart home management.
Business Rule	If a device is offline or unreachable, the app shall display an appropriate status message.
Dependencies	FR-3 (Device Onboarding)
Priority	High

1.3.5 Automatic Mode Activation (Functional Requirement: FR-5)

This requirement enables users to activate Automatic Mode, allowing the ESP32 to make decisions based on sensor inputs as shown in **Table 6**.

Table 6: Functional Requirement – Automatic Mode Activation

Identifier	FR-5
Title	Automatic Mode Activation
Requirement	The system shall allow the user to enable or disable Automatic Mode from the mobile application.
Source	Project Objective: Introduce Automatic Mode for environment-based decision-making.
Rationale	Allows the ESP32 to automatically control appliances using sensor inputs.
Business Rule	Automatic Mode overrides manual ON/OFF commands for supported appliances.
Dependencies	FR-3
Priority	High

1.3.6 Temperature-Based Fan Control (Functional Requirement: FR-6)

This requirement defines how the system uses temperature readings to automate fan control when Automatic Mode is active as shown in **Table 7**.

Table 7: Functional Requirement – Temperature Based Fan Control

Identifier	FR-6
Title	Temperature-Based Fan Control
Requirement	The system shall automatically turn the fan ON or OFF based on temperature thresholds when Automatic Mode is active.
Source	Project Goal: Enable intelligent automation using temperature sensors.
Rationale	Improves comfort and energy efficiency.
Business Rule	Threshold values may be predefined or user-adjustable.

Dependencies	FR-5
Priority	High

1.3.7 Motion-Based Light Automation (Functional Requirement: FR-7)

This requirement describes how the system uses motion detection to automate lighting as shown in **Table 8**.

Table 8: Functional Requirement – Motion-Based Light Automation

Identifier	FR-7
Title	Motion-Based Light Automation
Requirement	The system shall automatically turn lights ON or OFF based on motion detection when Automatic Mode is active.
Source	Project Goal: Enable automation using PIR motion sensors.
Rationale	Enhances convenience and reduces unnecessary energy usage.
Business Rule	Uses PIR sensor for occupancy detection.
Dependencies	FR-5
Priority	Medium

1.3.8 Light-Level Curtain Automation (Functional Requirement: FR-8)

As shown in **Table 9**, this requirement explains how the system uses ambient light levels to automate curtain movement.

Table 9: Functional Requirement – Light-Level Curtain Automation

Identifier	FR-8
Title	Light-Level Curtain Automation
Requirement	The system shall automatically open or close curtains based on ambient light levels when Automatic Mode is active.
Source	Project Goal: Enable automation using LDR sensors.

Rationale	Improves comfort and natural lighting efficiency.
Business Rule	Uses LDR sensor to detect day/night conditions.
Dependencies	FR-5
Priority	Medium

1.3.9 Water-Level Pump Automation (Functional Requirement: FR-9)

This requirement defines how the system manages water pump operation based on tank water levels as shown in **Table 10**.

Table 10: Functional Requirement – Water-Level Pump Automation

Identifier	FR-9
Title	Water-Level Pump Automation
Requirement	The system shall automatically turn the water pump ON or OFF based on tank water-level readings when Automatic Mode is active.
Source	Project Goal: Enable automation using water-level sensors.
Rationale	Prevents tank overflow and ensures efficient water usage.
Business Rule	Uses water-level sensor to detect minimum and maximum thresholds.
Dependencies	FR-5
Priority	High

1.3.10 Peak-Time Buzzer Alert (Functional Requirement: FR-10)

This requirement specifies how the system alerts users during predefined peak-time intervals as shown in **Table 11**.

Table 11: Functional Requirement – Peak-Timer Buzzer Alert

Identifier	FR-10
Title	Peak-Time Buzzer Alert

Requirement	The system shall activate a buzzer during predefined peak-time intervals.
Source	Project Goal: Provide user alerts based on time-based events.
Rationale	Helps users manage energy usage during peak hours.
Business Rule	Peak-time ranges may be predefined or user-adjustable.
Dependencies	FR-3
Priority	Low

1.3.11 Communication Handling (Functional Requirement: FR-11)

This requirement defines how the system manages communication between the mobile app, Firebase, and ESP32 as shown in **Table 12**.

Table 12: Communication Handling

Identifier	FR-11
Title	Communication Handling
Requirement	The system shall use Firebase Authentication for login and shall communicate with ESP32 devices using Wi-Fi and Firebase Realtime Database.
Source	Project Objective: Ensure seamless communication between the mobile app, Firebase, and ESP32 hardware.
Rationale	Ensures secure login and reliable device communication.
Business Rule	Device control shall occur through Firebase-synchronized commands.
Dependencies	FR-1, FR-2
Priority	High

1.3.12 Subscription Management (Functional Requirement: FR-12)

This requirement defines how the system handles subscription-based access. Users may have access to certain features only if they are subscribed. The system must allow users to view their subscription status and upgrade when needed as shown in

Table 13: Subscription management

Identifier	FR-12
Title	Subscription Management
Requirement	The system shall allow users to view their current subscription status and upgrade to a higher plan if required.
Source	Project Objective: Provide controlled access to selected features based on subscription level.
Rationale	Ensures that premium features (if any) are accessible only to subscribed users.
Business Rule	Subscription status must be verified through Firebase before granting access to restricted features.
Dependencies	FR-1 (Registration), FR-2 (Login)
Priority	Medium

1.4 Non-Functional Requirements

This section describes how well the system must perform rather than what it does. These requirements define the expected levels of reliability, usability, performance, and security for the IoT-Based Smart Home Automation System. The goals align with modern IoT consumer-grade standards for cloud-connected smart home systems. Performance, usability, reliability, and security goals reflect recommended IoT consumer-grade system expectations (Chen, Wang, & Zhang, 2025; Al-Fuqaha, Guibene, & Mohamed, 2015).

1.4.1 Reliability

The system must operate consistently and handle unexpected conditions gracefully to ensure dependable smart-home automation.

- **System Availability:** Firebase Authentication and Firebase Realtime Database must remain accessible to allow users to log in and synchronize device states. The system targets an availability of **99%**, excluding Firebase's scheduled maintenance.
- **Error Handling:** If the mobile app cannot communicate with the ESP32 or retrieve device status, the system should detect the issue and display an appropriate error message within **2 seconds**.

- **Power Loss Recovery:** After a power outage, the ESP32 should automatically restart, reconnect to Wi-Fi, and resume normal operation within **60 seconds**.
- **Sensor Reliability:** Sensors (DHT11, PIR, LDR, water-level) should provide consistent readings with at least **95% accuracy** to ensure correct automation behaviour.

1.4.2 Usability

The system must be intuitive, easy to learn, and efficient for daily use.

- **Basic Onboarding:** A new user should be able to install the app, create a Firebase account, and access the dashboard within **5 minutes**.
- **Quick Control:** Common actions such as turning devices ON/OFF or enabling Automatic Mode should require no more than **2–3 taps**.
- **Learnability:** A smartphone-literate user should understand the main features of the app within **10–15 minutes** without needing a manual.
- **Clarity in Feedback:** When invalid input is entered (e.g., non-numeric threshold values), the app must display a clear, helpful error message.

1.4.3 Performance

The system must respond quickly to user actions and sensor events to maintain a smooth experience.

- **Screen Response:** Screen transitions and button interactions should respond within **2 seconds**.
- **Speed of device control:** When a user toggles a device ON/OFF, the command should be processed and reflected in the app within **3 seconds** under normal network conditions.
- **Automation Response:** Automatic Mode actions (temperature-based fan control, motion-based lighting, curtain automation, water-level pump control) should activate within **2 seconds** of sensor input detection.
- **Voice Command Processing:** Device status changes should synchronize between Firebase and the app within **1–2 seconds**.

1.4.4 Security

The system must protect user data, prevent unauthorized access, and ensure secure communication.

- **Authentication Enforcement:** Only authenticated users (via Firebase Authentication) may access or control devices.
- **Information Security** All communication between the mobile app, Firebase, and ESP32 must use HTTPS encryption to prevent data interception.
- **Account Protection:** After **5 unsuccessful login attempts**, the account should be temporarily locked for **5 minutes** to prevent brute-force attacks.
- **User Isolation:** Each user must only have access to their own registered devices. No user should be able to view or control another user's devices.

1.5 External Interface Requirements

This section describes how the system interacts with external components such as the user, the mobile application, Firebase services, Wi-Fi communication, and the ESP32 hardware. The system architecture integrates a Flutter mobile app, Firebase cloud services, and an ESP32 microcontroller to deliver a unified smart-home automation experience. This design follows modern cloud-connected IoT models that link mobile clients, embedded hardware, and cloud middleware for reliable automation and real-time control. Architecture follows proven cloud-connected automation models linking mobile clients, embedded hardware, and cloud middleware (Firebase, 2024; Arduino, 2024).

1.6 User Interfaces Requirements

The system provides a mobile application that serves as the primary interface between the user and the smart-home environment. The UI must be simple, intuitive, and consistent with modern Android design guidelines to ensure accessibility and ease of use. The app should be straightforward and easy to use. Guidelines ensured based on standard Android development practices for accessibility and mobile consistency (Developers, 2024).

- **Look and Feel:** The application shall follow standard Android UI guidelines, using a clean colour palette, readable typography, and consistent spacing.
- **Layout:** The primary dashboard shall display all ESP32-connected devices. Tapping a device opens its control screen, where users can toggle ON/OFF, view status, or adjust automation settings. A menu button provides access to settings and logout.
- **Standard Items:** Each screen shall include a back button and clearly visible controls such as ON/OFF toggles, Automatic Mode switches, and sensor threshold inputs.
- **Notifications:** System notifications (e.g., *Automatic Mode Activated*, *Water Level Low*) shall be delivered using standard Android push notifications.
- **Clarity:** The UI shall be designed to support easy translation into other languages in future versions.

(Note: Exact screen designs and colours will be decided later and put in a separate design document.)

1.7 Software Interfaces

The system relies on several software components to enable authentication, data synchronization, and device communication.

- **Firebase Authentication:** Used for secure user registration and login. Provides reliable identity management suitable for IoT applications. Firebase offers real-time synchronization and secure authentication suitable for IoT automation (Firebase, 2024).

- **Firebase Realtime Database / Firestore:** Stores device metadata, user profiles, automation settings, and real-time device states.
- **Firebase Cloud Messaging (FCM):** Sends push notifications to the user's mobile device for alerts such as automation triggers or peak-time buzzer activation.
- **Local Network Communication:** The mobile app communicates with the ESP32 indirectly through Firebase. The ESP32 connects to Wi-Fi and synchronizes device states and commands using HTTP or MQTT (implementation dependent).

1.8 Hardware Interfaces

The system interacts with physical devices through the ESP32 microcontroller and connected sensors and actuators. Arduino-based control with relay switching remains most preferred approach for low-cost smart home deployment (Sharma, 2022; Arduino, 2024).

- **ESP32 Controller:** The primary hardware controller responsible for reading sensor data, executing automation logic, and controlling appliances through relays.
- **Relay Modules:** Used to switch AC appliances such as fans, lights, curtains, and water pumps.
- **Sensors:**
 - **DHT11:** Measures temperature for fan automation.
 - **LDR:** Detects ambient light levels for curtain/light automation.
 - **PIR Sensor:** Detects motion for light automation.
 - **Water-Level Sensor:** Monitors tank levels for pump control.
- **Buzzer:** Used to generate peak-time alerts or system notifications.

1.9 Communications Interfaces

The system uses secure and efficient communication channels between the mobile app, Firebase cloud, and ESP32 hardware. HTTPS and JSON formats are recommended for IoT cloud interoperability and secure message exchange (Al-Fuqaha, Guibene, & Mohamed, 2015).

- **Primary Protocol:**
 - Firebase communication uses **HTTPS** for secure data transfer.
 - ESP32 device communication uses HTTP or MQTT, depending on implementation.
- **Data Format:** All messages exchanged between the mobile app, Firebase, and ESP32 shall use **JSON** format for interoperability and simplicity.
- **Network Requirement:** Internet access is required for Firebase operations such as login and device synchronization. The ESP32 must be connected to the same Wi-Fi network during initial setup. After onboarding, device control can occur through cloud synchronization even if the phone and ESP32 are on different networks.

2. Use Case Analysis

This section breaks down the main ways users will interact with the system. Each "use case" is a specific goal a user wants to achieve.

2.1 Use Case #1: User Registration (FR-1)

This use case allows a new user to create an account by providing their basic credentials so they can securely access the system as shown in Figure 1 & Table 14.

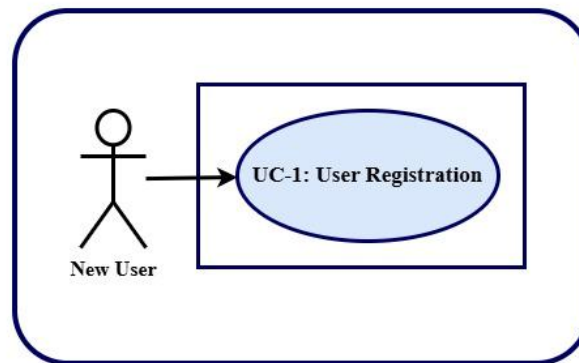


Figure 1: User Registration (FR-1)

Following is the table for UC-1:

Table 14: Use Case Description for "User Registration"

UC Identifier	UC-1
Requirements Traceability	FR-1
Purpose	Allow a new user to create an account securely.
Priority	High
Preconditions	App installed; user not logged in.
Post conditions	Account created in Firebase.
Actors	New User
Extends	--
Main Success Scenario	• User opens app → selects "Sign Up". • Enters email + password. • System validates credentials. • Account created. • User redirected to Login screen.
Alternate Flows	A1: User enters weak password - app suggests stronger password.
Exceptions	E1: Email already registered - shows error.
Includes	--

2.2 Use Case #2: Login to the System (FR-2)

This use case allows a registered user to sign into the system by entering valid login credentials to access their connected devices and app features as shown in Figure 2 & Table 15.

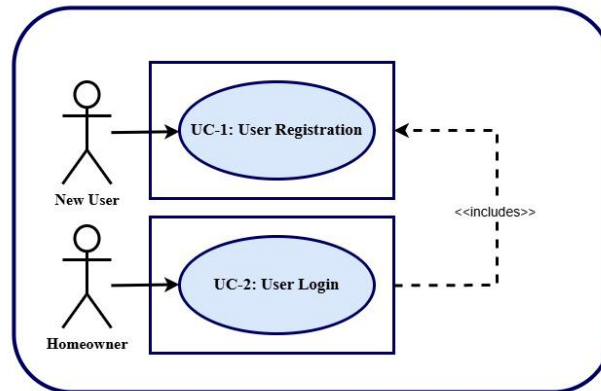


Figure 2: Login to the System (FR-2)

Following is the table for UC-2:

Table 15: Use Case Description for " Login to the System "

UC Identifier	UC-2
Requirements Traceability	FR-2
Purpose	Verify user credentials and allow secure access to the dashboard.
Priority	High
Preconditions	Valid registered account exists.
Post conditions	User enters app dashboard.
Actors	Homeowner / Resident
Extends	--
Main Success Scenario	• User opens app. • Enters email & password. • System verifies. • User logged in successfully.
Alternate Flows	A1: User selects Forgot Password to reset credentials.
Exceptions	E1: Incorrect password - show error. E2: After 5 failed attempts - temporary lock.
Includes	UC-1 (only when user chooses "Sign Up")

2.3 Use Case #3: Add/Pair a New Device (ESP32) (FR-3)

This use case enables the user to connect a new smart appliance (ESP32) to their account through a guided setup process in the mobile app as shown in Figure 3 & Table 16.

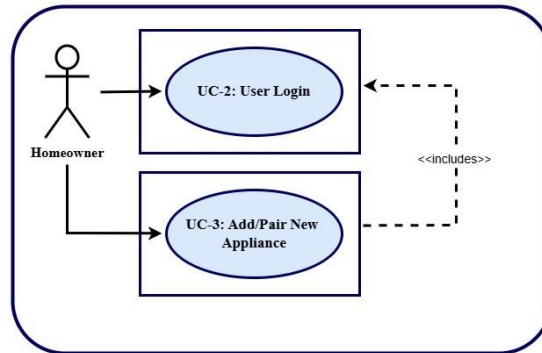


Figure 3: Add/Pair an Appliance (FR-3)

Following is the table for UC-3:

Table 16: Use Case Description for "Adding/Pairing a new appliance"

UC Identifier	UC-3
Requirements Traceability	FR-3
Purpose	Allow user to add ESP32-controlled devices.
Priority	High
Preconditions	User logged in; ESP32 powered and connected to Wi-Fi.
Post conditions	Device appears in dashboard.
Actors	Homeowner / Resident
Extends	--
Main Success Scenario	• User selects Add Device. • System retrieves available devices from Firebase. • User selects device. • Device linked to account. • Device added to dashboard.
Alternate Flows	A1: User enters Device ID manually.
Exceptions	E1: Device not found - prompts user to retry.
Includes	UC-2

2.4 Use Case #4: Control Device ON/OFF (FR-4)

This use case allows the user to switch a connected Wi-Fi appliance ON or OFF using the mobile application as shown in Figure 4 & Table 17.

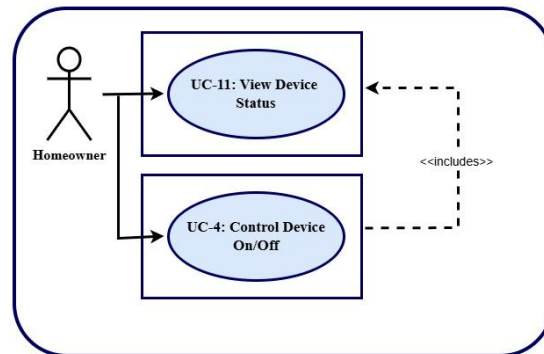


Figure 4: Control Device ON/OFF (FR-4)

Following is the table for UC-4:

Table 17: Use Case Description for " Controlling Device "

UC Identifier	UC-4
Requirements Traceability	FR-4
Purpose	Allow user to toggle appliances ON/OFF.
Priority	High
Preconditions	Device paired and reachable.
Post conditions	Device state updated.
Actors	Homeowner / Resident
Extends	--
Main Success Scenario	▪ User taps toggle button. ▪ Command sent to Firebase. ▪ ESP32 receives update. ▪ Device switches state. ▪ App updates status.
Alternate Flows	--
Exceptions	E1: Device unreachable
Includes	UC-11 (viewing device status)

2.5 Use Case #5: Activate Automatic Mode (FR-5)

This use case allows the user to enable or disable Automatic Mode as shown in Figure 5 and Table 18.

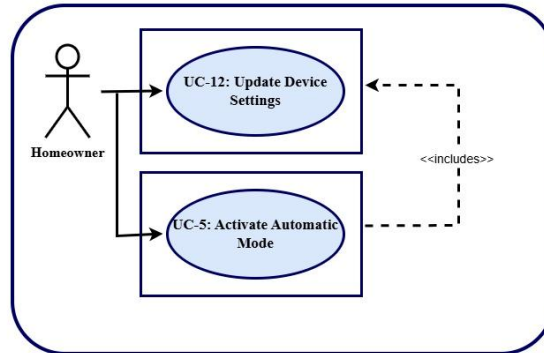


Figure 5: Automatic Mode Activation (FR-5)

Following is the table for UC-5:

Table 18: Use Case Description for " Automatic Mode Activation "

UC Identifier	UC-5
Requirements Traceability	FR-5
Purpose	Allow user to enable/disable Automatic Mode.
Priority	High
Preconditions	Device paired; sensors connected.
Post conditions	Automatic Mode activated.
Actors	Homeowner / Resident
Extends	--
Main Success Scenario	• User opens device settings. • Toggles “Automatic Mode” • Firebase updates mode. • ESP32 begins automation.
Alternate Flows	--
Exceptions	E1: ESP32 not responding.
Includes	UC-12 (when adjusting thresholds)

2.6 Use Case #6: Temperature-Based Fan Automation (FR-6)

This use case describes how the ESP32 automatically controls the fan based on temperature.as shown in Figure 6 and Table 19.

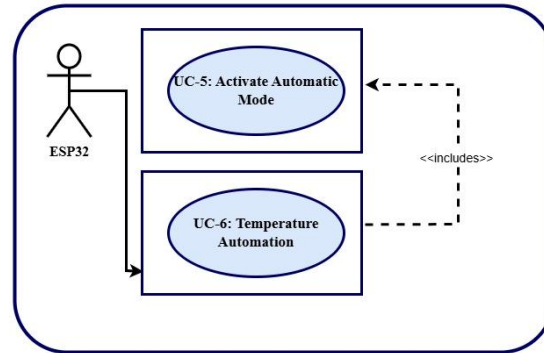


Figure 6: Temperature-Based Fan Automation (FR-6)

Following is the table for UC-6:

Table 19: Use Case Description for " Temperature-Based Fan Automation "

UC Identifier	UC-6
Requirements Traceability	FR-6
Purpose	Automatically control fan based on temperature.
Priority	High
Preconditions	Automatic Mode enabled; DHT11 connected.
Post conditions	Fan ON/OFF based on threshold.
Actors	ESP32 (System)
Extends	--
Main Success Scenario	• ESP32 reads temperature. • If temperature > threshold → Fan ON. • If temperature < threshold → Fan OFF.
Alternate Flows	--
Exceptions	E1: Sensor malfunction.
Includes	UC-5

2.7 Use Case #7: Motion-Based Light Automation (FR-7)

This use case describes how the ESP32 controls lights based on motion detection as shown in Figure 7 & Table 20.

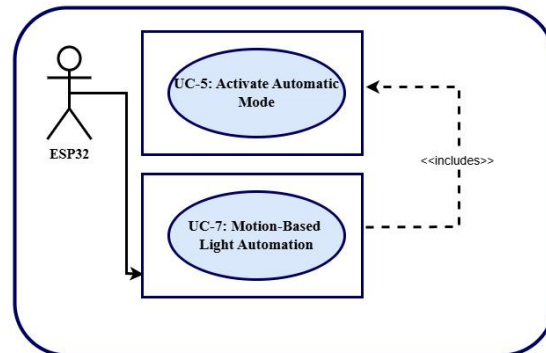


Figure 7: Motion-Based Light Automation (FR-7)

Following is the table for UC-7:

Table 20: Use Case Description for " Motion-Based Light Automation "

UC Identifier	UC-7
Requirements Traceability	FR-7
Purpose	Automatically control lights based on motion.
Priority	Medium
Preconditions	Automatic Mode enabled; PIR sensor connected.
Post conditions	Lights ON/OFF automatically.
Actors	ESP32 (System)
Extends	--
Main Success Scenario	• ESP32 detects motion. • Light turns ON. • No motion for set duration → Light OFF.
Alternate Flows	--
Exceptions	E1: Sensor not responding.
Includes	UC-5

2.8 Use Case #8: Light-Level Curtain Automation (FR-8)

This use case describes how the ESP32 automates curtains based on ambient light.as shown in Figure 8 & Table 21.

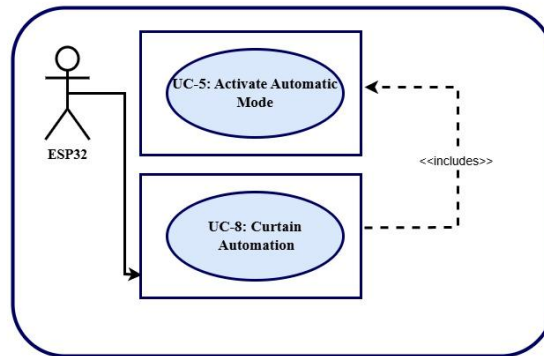


Figure 8: Light-Level Curtain Automation (FR-8)

Following is the table for UC-8:

Table 21: Use Case Description for " Light-Level Curtain Automation "

UC Identifier	UC-8
Requirements Traceability	FR-8
Purpose	Automatically open/close curtains based on light level.
Priority	Medium
Preconditions	Automatic Mode enabled; LDR connected.
Post conditions	Curtains open/close automatically.
Actors	ESP32 (System)
Extends	--
Main Success Scenario	• ESP32 reads light level. • Low light → Curtains open. • High light → Curtains close.
Alternate Flows	--
Exceptions	E1: Sensor malfunction.
Includes	UC-5

2.9 Use Case #9: Water-Level Pump Automation (FR-9)

This use case describes how the ESP32 controls the water pump based on tank level.as shown in Figure 9 and Table 22.

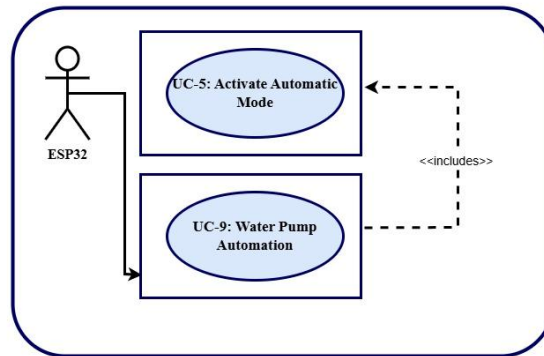


Figure 9: Water-Level Pump Automation (FR-9)

Following is the table for UC-9:

Table 22: Use Case Description for " Water-Level Pump Automation "

UC Identifier	UC-9
Requirements Traceability	FR-9
Purpose	Automatically control pump based on water level.
Priority	High
Preconditions	Automatic Mode enabled; water-level sensor connected.
Post conditions	Pump ON/OFF automatically.
Actors	ESP32 (System)
Extends	--
Main Success Scenario	• ESP32 reads water level. • Low level → Pump ON. • High level → Pump OFF.
Alternate Flows	--
Exceptions	E1: Sensor malfunction.
Includes	UC-5

2.10 Use Case #10: Peak-Time Buzzer Alert (FR-10)

This use case describes how the ESP32 triggers a buzzer during peak hours.as shown in Figure 10 & Table 23.

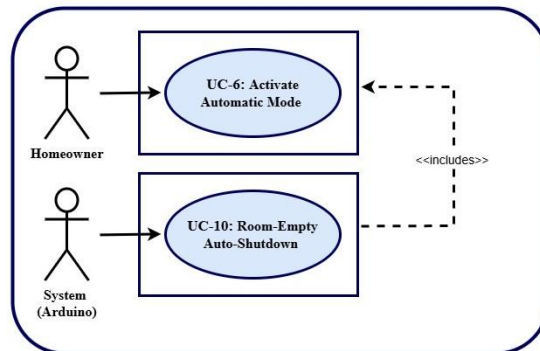


Figure 10: Peak-Time Alert (FR-10)

Following is the table for UC-10:

Table 23: Use Case Description for " Peak-Time Alert "

UC Identifier	UC-10
Requirements Traceability	FR-10
Purpose	Notify users during peak-time intervals.
Priority	Low
Preconditions	ESP32 connected; peak-time configured.
Post conditions	Buzzer activated.
Actors	ESP32 (System)
Extends	--
Main Success Scenario	• System clock reaches peak time. • ESP32 activates buzzer. • App displays notification.
Alternate Flows	--
Exceptions	E1: Buzzer not responding.
Includes	-

2.11 Use Case #11: View Device Status (FR-11)

This use case allows user to view real-time status of all devices as shown in Figure 11 & Table 24.

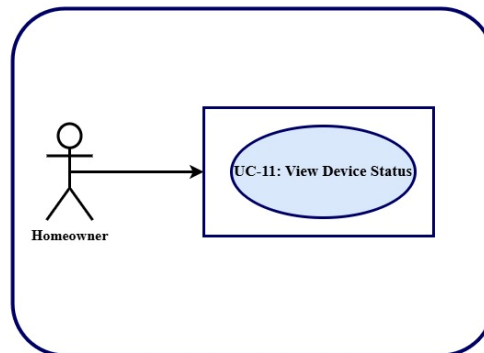


Figure 11: View Device Status (FR-11)

Following is the table for UC-11:

Table 24: Use Case Description for " View Device Status"

UC Identifier	UC-11
Requirements Traceability	FR-11
Purpose	Display real-time device states.
Priority	Medium
Preconditions	Devices paired.
Post conditions	Status displayed.
Actors	Homeowner / Resident
Extends	--
Main Success Scenario	• User opens dashboard. • App fetches device states from Firebase. • States displayed.
Alternate Flows	--
Exceptions	E1: Device unreachable.
Includes	--

2.12 Use Case #12: Update Device Settings (FR-5, FR-6, FR-7, FR-8, FR-9)

This use case allows the user to update automation thresholds and settings.as shown in Figure 12 & Table 25.

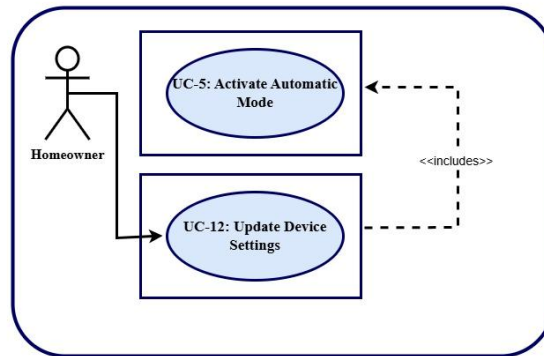


Figure 12: Update Device Settings

Following is the table for UC-12:

Table 25: Use Case Description for "Update Device Settings"

UC Identifier	UC-12
Requirements Traceability	FR-5, FR-6, FR-7, FR-8, FR-9
Purpose	Allow user to update thresholds and automation settings.
Priority	Medium
Preconditions	Device paired; user logged in.
Post conditions	New settings saved and applied.
Actors	Homeowner / Resident
Extends	--
Main Success Scenario	• User opens device settings. • Adjusts thresholds or modes. • App sends updated settings to Firebase. • ESP32 applies new configuration.
Alternate Flows	--
Exceptions	E1: Invalid input values.
Includes	UC-5

2.13 Use Case #13: Manage Subscription (FR-12)

This use case allows the user to view their subscription status and upgrade to a higher plan if required. It does not introduce new system features; it simply manages access to existing ones based on subscription level as shown in Figure 13 &

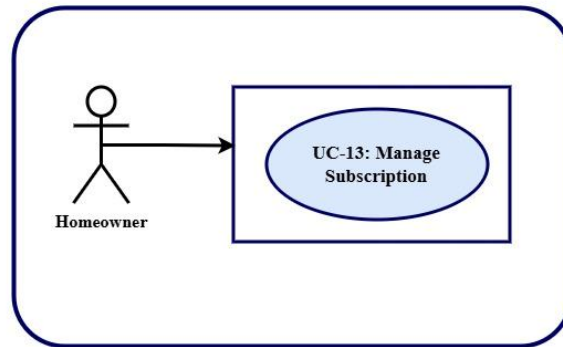


Figure 13: Manage Subscription (FR-12)

Following is the table for UC-13:

Table 26: Use Case Description for " Manage Subscription "

UC Identifier	UC-12
Requirements Traceability	FR-5, FR-6, FR-7, FR-8, FR-9
Purpose	Allow user to update thresholds and automation settings.
Priority	Medium
Preconditions	Device paired; user logged in.
Post conditions	New settings saved and applied.
Actors	Homeowner / Resident
Extends	--
Main Success Scenario	• User opens device settings. • Adjusts thresholds or modes. • App sends updated settings to Firebase. • ESP32 applies new configuration.
Alternate Flows	--
Exceptions	E1: Invalid input values.
Includes	UC-5

2.14 Use Case Diagram

Figure 14 illustrates how users interact with the IoT-Based Smart Home Automation System. A new user can register an account and, once authenticated, becomes a Homeowner who can log in, add ESP32-controlled devices, and manage them through the mobile application. After onboarding, the Homeowner can perform core actions such as toggling appliances ON/OFF, enabling Automatic Mode, viewing real-time device status, and updating automation settings. The system also supports intelligent behaviours

including temperature-based fan control, motion-based light automation, light-level curtain automation, water-level pump automation, and peak-time buzzer alerts. Together, these interactions represent the complete functional scope of the system.

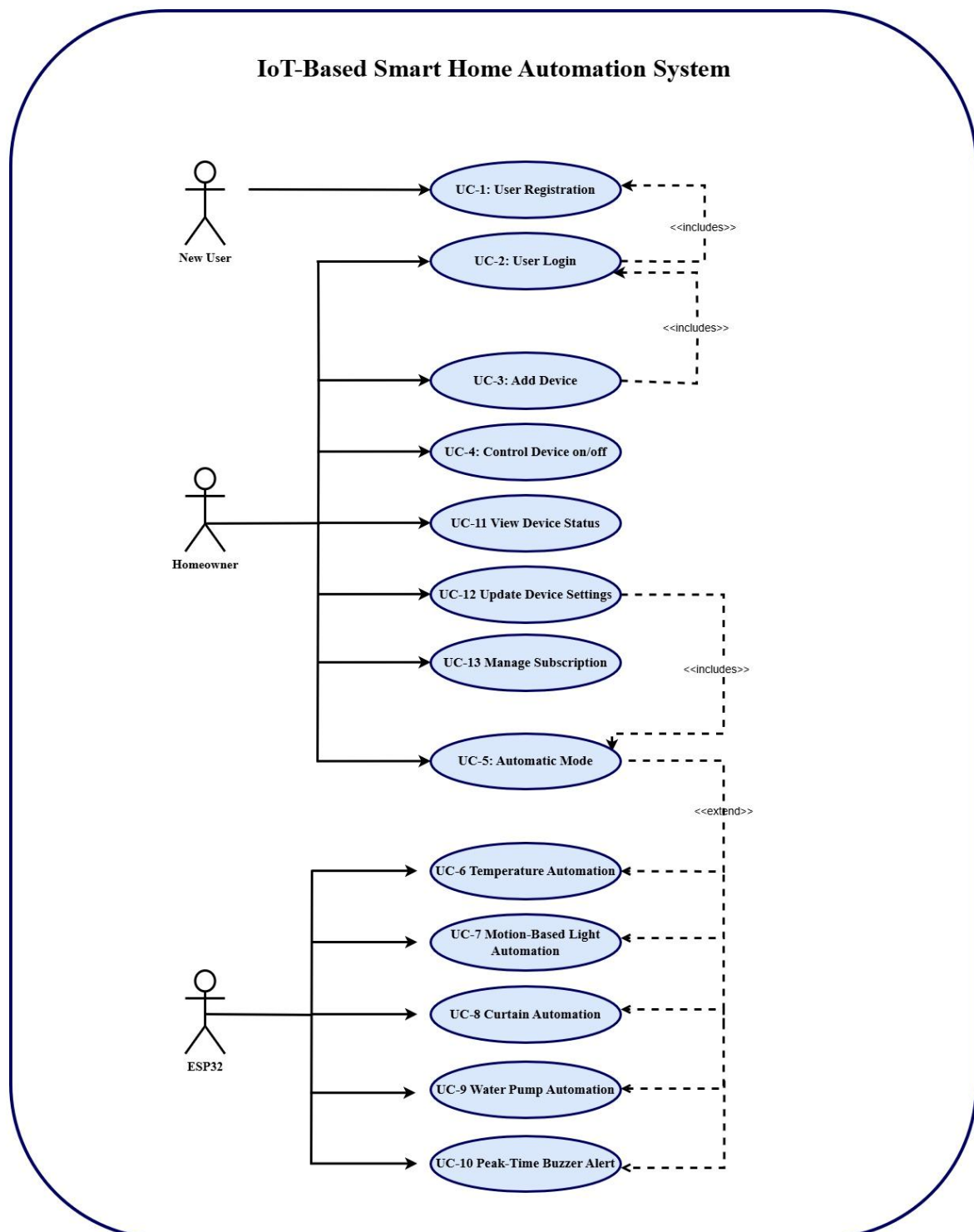


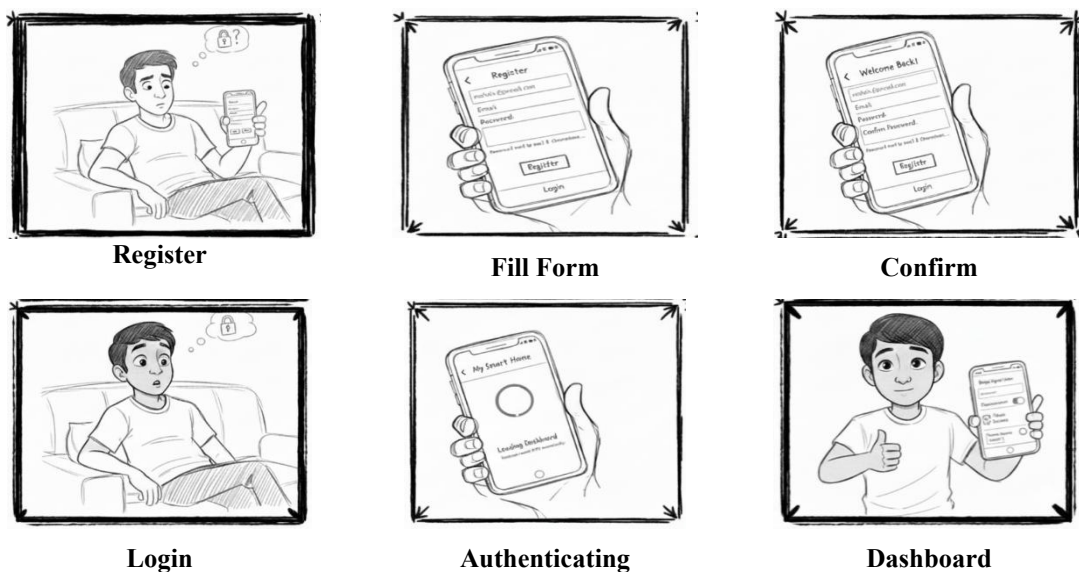
Figure 14: Use Case Diagram

3. Storyboards

Storyboards are sequence of drawings representing how users might use the product. They show user steps and corresponding responses by the system.

3.1 Storyboard 1: Registration, Login, and Dashboard Flow

The following storyboard illustrates the initial user journey within the smart home application. It focuses on the core entry-point screens, including account registration, user login, and the main dashboard interface. These screens represent the foundational steps a user completes before accessing device control, automation features, or voice commands.



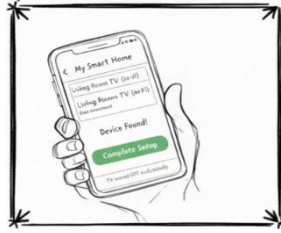
3.2 Storyboard 2: Device Pairing and Control

This storyboard illustrates the user journey for selecting, pairing, and controlling a Wi-Fi device (TV) through the smart home mobile application. It highlights the interaction between the mobile app and the smart device, including system feedback and user actions.





User reacts to pairing prompt



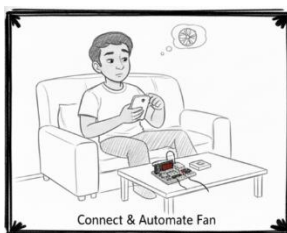
TV paired — setup complete



Control options now available

3.3 Storyboard 3: Fan Automation (Temperature Sensor)

This storyboard shows how the user connects a fan through Arduino, enables Automatic Mode in the app, and lets the system adjust fan speed automatically based on room temperature.



User connects fan to Arduino



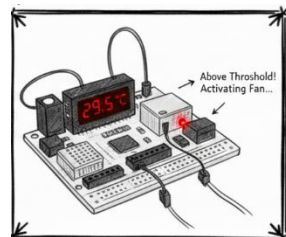
User opens dashboard



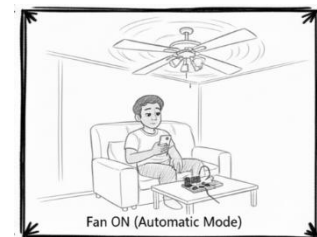
Automatic Mode enabled



Temperature exceeds threshold



Fan speed increased



Comfort maintained automatically

4. Summary

Requirement analysis focused on identifying the real needs of a household user and converting them into precise system specifications. Observations of daily appliance usage and informal discussions highlighted challenges such as managing multiple appliances, lack of automation, and the need for remote control. These insights shaped both the functional and non-functional requirements of the system, including Firebase-based authentication, Wi-Fi-enabled ESP32 devices, Automatic Mode, and sensor-driven automation for temperature, motion, light level, water level, and peak-time alerts.

Use case analysis structured the system's behaviour by modelling every interaction between the user and the smart home application—from registration and login to device pairing, manual control, automation configuration, and subscription management. The individual use case diagrams, scenario flows, and interaction modelling clarified how the mobile app communicates with Firebase and how ESP32 modules execute automation logic based on real-time sensor data. These structured requirements engineering approach reduced ambiguity and ensured a shared understanding among stakeholders.

The resulting specification serves as a complete blueprint for implementing a unified smart home solution. It supports remote ON/OFF control, real-time device monitoring, and fully automated behaviour through ESP32-based sensors and actuators. Through systematic requirement analysis, the project evolved from a general concept into a technically defined smart home automation architecture ready for implementation.

5. References

- Ahmad, S. K. (2023). Cloud-Based Smart Home Management System for Energy Optimization. *Springer Nature Computer Science*, 4(3), 90-101.
- Al-Fuqaha, A., Guibene, M., & Mohamed, N. (2015). Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications. *IEEE Communications Surveys & Tutorials*, 17(4), 2347–2376.
- Arduino. (2024). *Arduino Official Documentation*. Retrieved November 10, 2024 from <https://docs.arduino.cc/>
- Association, I. S. (2025). *IEEE P2890—Standard for Smart Home Energy Efficiency and Interoperability*.
- Chen, M., Wang, L., & Zhang, H. (2025). Edge-AI for Real-Time IoT Automation: Reducing Latency in Smart Home Systems. *IEEE Internet of Things Journal*, 4500–4515.
- Developers, G. (2024, Novemberx 10). *Flutter Documentation*. From <https://docs.flutter.dev/>
- Firebase, G. (2024). *Firebase Documentation*. Retrieved November 10, 2024 from <https://firebase.google.com/docs>
- Kaur, G. &. (2020). A Review on Smart Home Automation Techniques Using IoT. *Journal of Ambient Intelligence and Humanized Computing*, 11(8), 3465–3478.
- Khan, A. R., & Singh, S. P. (2025). Federated Learning for Privacy-Preserving Smart Home Energy Management. *ACM Transactions on Internet of Things*, 112–130.
- Nguyen, T., Patel, B., & Kumar, R. (2025). Low-Cost Arduino-Based Smart Plugs with Cloud-Integrated Energy Monitoring. *Proc. IEEE International Conference on Consumer Electronics (ICCE)*, (pp. 1–6). Las Vegas.
- Sharma, R. &. (2022). IoT-Based Smart Home Automation System Using Arduino and Mobile Application. *International Journal of Computer Applications*, 183(22), 15–20.
- Sommerville, I. (2016). *Software Engineering, 10th Edition*. Harlow: Pearson Education.
- TensorFlow. (2024). *TensorFlow Lite for Microcontrollers*. Retrieved November 10, 2024 from <https://www.tensorflow.org/lite/microcontrollers>